

Assignment :- 6
Parallel and Distributed Computing
Submitted by
Bhavish Pushkarna (102303279)

Q1: Write the device query code, compile and run it on your system. Query enough information to know all the details of your device. Example queries: GPU card's name, GPU computation capabilities, Maximum number of block dimensions, Maximum number of grid dimensions, Maximum size of GPU memory, Amount of constant and share memory, Warp size, etc. Answer the following questions in your report.

1. What is the architecture and compute capability of your GPU?
2. What are the maximum block dimensions for your GPU?
3. Suppose you are launching a one dimensional grid and block. If the hardware's maximum grid dimension is 65535 and the maximum block dimension is 512, what is the maximum number threads can be launched on the GPU?
4. Under what conditions might a programmer choose not want to launch the maximum number of threads?
5. What can limit a program from launching the maximum number of threads on a GPU?
6. What is shared memory? How much shared memory is on your GPU?
7. What is global memory? How much global memory is on your GPU?
8. What is constant memory? How much constant memory is on your GPU?
9. What does warp size signify on a GPU? What is your GPU's warp size?
10. Is double precision supported on your GPU?

Code:

```
LAB6 x G device_query.cu x
1 #include <iostream>
2 #include <cuda_runtime.h>
3
4 int main() {
5     int deviceCount;
6     cudaGetDeviceCount(&deviceCount);
7
8     if (deviceCount == 0) {
9         std::cout << "No CUDA-capable devices found." << std::endl;
10        return 1;
11    }
12
13    for (int i = 0; i < deviceCount; ++i) {
14        cudaDeviceProp prop;
15        cudaGetDeviceProperties(&prop, i);
16
17        std::cout << "Device Number: " << i << std::endl;
18        std::cout << " Device Name: " << prop.name << std::endl;
19        std::cout << " Compute Capability: " << prop.major << "." << prop.minor << std::endl;
20        std::cout << " Max Block Dimensions: [" << prop.maxThreadsDim[0] << ", " << prop.maxThreadsDim[1] << ", " << prop.maxThreadsDim[2] << "]" << std::endl;
21        std::cout << " Max Grid Dimensions: [" << prop.maxGridSize[0] << ", " << prop.maxGridSize[1] << ", " << prop.maxGridSize[2] << "]" << std::endl;
22        std::cout << " Total Global Memory: " << prop.totalGlobalMem / (1024 * 1024) << " MB" << std::endl;
23        std::cout << " Shared Memory per Block: " << prop.sharedMemPerBlock / 1024 << " KB" << std::endl;
24        std::cout << " Total Constant Memory: " << prop.totalConstMem / 1024 << " KB" << std::endl;
25        std::cout << " Warp Size: " << prop.warpSize << std::endl;
26        .....
27    }
28    return 0;
29 }
30 }
```

Output:

```
Device Name: NVIDIA GeForce GTX 1650 Ti
Compute Capability: 7.5
Max Block Dimensions: [1024, 1024, 64]
Max Grid Dimensions: [2147483647, 65535, 65535]
Total Global Memory: 3715 MB
Shared Memory per Block: 48 KB
Total Constant Memory: 64 KB
Warp Size: 32
```

Answers to Asked Question :

Answer 1. Architecture: Turing Compute Capability: 7.5

Answer 2. Maximum block dimensions: (1024, 1024, 64)

Answer 3. $65535 \times 512 = 33,553,920$ threads

Answer 4.

- If the dataset is smaller than the maximum thread count.
- If each thread requires a large amount of local resources registers or shared memory, launching maximum threads would cause register spilling or fail to launch entirely.
- To optimize cache hits and memory bandwidth (too many threads can thrash the cache).

Answer 5. Registers per block: If a kernel uses too many variables, it hits the register limit before the thread limit.

Shared memory per block: If threads request too much shared memory, fewer blocks can be scheduled on a Streaming Multiprocessor (SM).

Answer 6. Shared Memory is an extremely fast, on-chip programmable cache that is shared strictly among threads within the same block. It is used for inter-thread communication and data reuse.

Amount: 49152 bytes (48 KB) per block

Answer 7. Global memory is the main, largest memory space on the GPU. It is accessible by all threads across all blocks, but it has high latency (slow) compared to shared memory.

Amount: 3.62 GB

Answer 8. Constant memory is a small, read-only memory space cached on-chip. It is optimized for situations where all threads in a warp read the exact same memory address simultaneously.

Amount: 64 KB

Answer 9. It is the fundamental unit of execution on an NVIDIA GPU. The hardware executes instructions in groups of threads called warps. Threads within a warp execute in lockstep (SIMT: Single Instruction, Multiple Threads).

Size: 32

Answer 10. Yes, almost all modern GPUs (Compute Capability 1.3 and above) support double precision (64-bit floating point), though the ratio of double-precision to single-precision performance varies heavily by architecture.

Q2: Write a CUDA program to calculate the sum of the elements in an array. The array contains single precision floating point numbers. Generate your input array. For this question, do the following.

1. Allocate device memory
2. Copy host memory to device
3. Initialize thread block and kernel grid dimensions
4. Invoke CUDA kernel
5. Copy results from device to host
6. Free device memory
7. Write the CUDA kernel that computes the sum.

Code:

```
1 #include <iostream>
2 #include <vector>
3 #include <cuda_runtime.h>
4
5 __global__ void sumKernel(float* d_array, float* d_sum, int N) {
6     int idx = blockIdx.x * blockDim.x + threadIdx.x;
7     if (idx < N) {
8         atomicAdd(d_sum, d_array[idx]);
9     }
10 }
11
12 int main() {
13     int N = 10000;
14     size_t size = N * sizeof(float);
15     ...
16     std::vector<float> h_array(N, 1.5f);
17     float h_sum = 0.0f;
18
19     float *d_array, *d_sum;
20     cudaMalloc((void**)&d_array, size);
21     cudaMalloc((void**)&d_sum, sizeof(float));
22
23     cudaMemcpy(d_sum, 0, sizeof(float));
24
25     cudaMemcpy(d_array, h_array.data(), size, cudaMemcpyHostToDevice);
26
27     int threadsPerBlock = 256;
28     int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
29
30     sumKernel<<<blocksPerGrid, threadsPerBlock>>>(d_array, d_sum, N);
31     cudaDeviceSynchronize();
32     cudaMemcpy(&h_sum, d_sum, sizeof(float), cudaMemcpyDeviceToHost);
33
34     std::cout << "Total Sum: " << h_sum << std::endl;
35     cudaFree(d_array);
36     cudaFree(d_sum);
37
38     return 0;
39 }
40 }
```

Output:

```
all_might ~/Documents/UCS645/LAB6 git-[P LAB3]- >> ./array_sum
Total Sum: 15000
```

Q3: Perform Matrix Addition of two large integer matrices in CUDA.

Answer the following questions.

1. How many floating operations are being performed in the matrix addition kernel?
2. How many global memory reads are being performed by your kernel?
3. How many global memory writes are being performed by your kernel?

Code:

```
LAB6 x C:\matrix_addition.cu x
42 #include <iostream>
41 #include <vector>
40 #include <cuda_runtime.h>
39
38 __global__ void matAddKernel(int* A, int* B, int* C, int width, int height) {
37     int col = blockIdx.x * blockDim.x + threadIdx.x;
36     int row = blockIdx.y * blockDim.y + threadIdx.y;
35
34     if (col < width && row < height) {
33         int idx = row * width + col;
32         C[idx] = A[idx] + B[idx];
31     }
30 }
29
28 int main() {
27     int width = 1024, height = 1024;
26     size_t size = width * height * sizeof(int);
25
24     std::vector<int> h_A(width * height, 1);
23     std::vector<int> h_B(width * height, 2);
22     std::vector<int> h_C(width * height, 0);
21
20     int *d_A, *d_B, *d_C;
19     cudaMalloc(&d_A, size);
18     cudaMalloc(&d_B, size);
17     cudaMalloc(&d_C, size);
16
15     cudaMemcpy(d_A, h_A.data(), size, cudaMemcpyHostToDevice);
14     cudaMemcpy(d_B, h_B.data(), size, cudaMemcpyHostToDevice);
13
12     dim3 threadsPerBlock(16, 16);
11     dim3 blocksPerGrid((width + threadsPerBlock.x - 1) / threadsPerBlock.x,
10                     (height + threadsPerBlock.y - 1) / threadsPerBlock.y);
9
8     matAddKernel<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, width, height);
7
6     cudaMemcpy(h_C.data(), d_C, size, cudaMemcpyDeviceToHost);
5
4     std::cout << "Top-left element of result matrix: " << h_C[0] << " (Expected 3)" << std::endl;
3
2     cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
1     return 0;
43 }
```

Output:

```
all_might ~/Documents/UCS645/LAB6 git-[LAB3]- >> ./matrix_add
Top-left element of result matrix: 3 (Expected 3)
```

Answers to asked Questions:

1. **Floating operations:** Zero, Because the prompt specified "two large *integer* matrices," the arithmetic operation ($A[idx] + B[idx]$) uses integer addition, not floating-point operations (FLOP's).
2. **Global memory reads:** For an $N \times M$ matrix, every thread reads one element from matrix A and one element from matrix B.
Total reads = $2 \times N \times M$.
3. **Global memory writes:** Every thread writes one computed element to the result matrix C.
Total writes = $N \times M$.